Enumeration Techniques Overview

This PDF describes the 10 main virtual-enumeration techniques selected for the USP5 project and how each one would be carried out with Python.

The techniques are:
- R-Group Enumeration
- Reaction-Based Enumeration
- Reagent-Pool Combinatorics
- Matched Molecular Pair Expansion
- Bioisosteric Swaps
- Linker Scans
- Ring-Size And Ring-System Scans
- Heteroatom Walks
- Scaffold Hopping
- Fragment Growing

These are implementation-oriented descriptions, not full production code. The idea is to show the role of each technique and the Python workflow behind it.

1. R-Group Enumeration

What it is: Keep a core scaffold fixed and systematically substitute allowed groups at one or more attachment points.

Why use it: This is the most direct way to explore local SAR around a known active core.

How it would be done with Python:
- Represent the scaffold with labeled attachment atoms such as [*:1], [*:2].
- Store allowed substituent pools as SMILES strings or reaction-ready fragments.
- Loop over all allowed combinations and attach them to the scaffold.
- Canonicalize, deduplicate, and record the parent scaffold plus substituent IDs.

Minimal pseudocode:

```
    scaffold = 'core-[*:1]-[*:2]'
    r1_pool = ['F', 'Cl', 'CN']
    r2_pool = ['piperidine', 'morpholine', 'pyrrolidine']
    for r1 in r1_pool:
        for r2 in r2_pool:
            analog = attach_groups(scaffold, {1: r1, 2: r2})
            save(analog)
```

2. Reaction-Based Enumeration

What it is: Generate compounds only through plausible synthetic reactions such as amide coupling, reductive amination, or Suzuki coupling.

Why use it: This keeps the library grounded in chemistry that can actually be made.

How it would be done with Python:
- Define reactions as transforms or SMARTS-like templates.
- Load reagent lists for each reaction partner.
- Apply each reaction to compatible reagent pairs.
- Filter invalid products and annotate the reaction used.

Minimal pseudocode:

```
    amide_rxn = define_reaction('acid + amine -> amide')
    for acid in acid_pool:
        for amine in amine_pool:
            product = run_reaction(amide_rxn, acid, amine)
            if product is not None:
                save(product, route='amide_coupling')
```

3. Reagent-Pool Combinatorics

What it is: Build large libraries by crossing compatible sets of reagents across one or more steps.

Why use it: This is the fastest route from a few parent chemotypes to thousands of virtual analogs.

How it would be done with Python:
- Create reagent tables grouped by role such as acids, amines, aryl halides, boronic acids.
- Define which pools can be crossed in each step.
- Enumerate all compatible combinations.
- Track reagent lineage so every product can be traced back to building blocks.

Minimal pseudocode:

```
    for acid in acid_pool:
        for amine in amine_pool:
            intermediate = couple(acid, amine)
            for aryl_halide in aryl_halide_pool:
                final = diversify(intermediate, aryl_halide)
                save(final)
```

4. Matched Molecular Pair Expansion

What it is: Apply small medicinal chemistry edits that are known to change potency, polarity, or selectivity in interpretable ways.

Why use it: This explores nearby chemical space without drifting too far from active parents.

How it would be done with Python:
- Create a library of local transforms such as F to CN, phenyl to pyridyl, methyl to cyclopropyl.
- Search each parent molecule for transformable sites.
- Apply one change at a time to generate local analogs.
- Record the exact transform used for later SAR analysis.

Minimal pseudocode:

```
    transforms = [('F', 'CN'), ('phenyl', 'pyridyl')]
    for mol in parents:
        for old, new in transforms:
            for analog in apply_local_transform(mol, old, new):
                save(analog, transform=f'{old}->{new}')
```

5. Bioisosteric Swaps

What it is: Replace a functional group with a chemically different group that can play a similar role in binding or properties.

Why use it: Bioisosteres are a standard way to improve potency, stability, permeability, or safety.

How it would be done with Python:
- Build dictionaries of common bioisostere replacements.
- Match eligible motifs in each molecule.
- Substitute the motif while preserving attachment geometry where possible.
- Recompute descriptors so the property effect is visible.

Minimal pseudocode:

```
    bioisosteres = {'carboxylic_acid': ['tetrazole', 'acylsulfonamide']}
    for mol in parents:
        for motif, replacements in bioisosteres.items():
            for repl in replacements:
                analogs = swap_motif(mol, motif, repl)
                save_all(analogs)
```

6. Linker Scans

What it is: Change the connector between two motifs by varying length, flexibility, saturation, or heteroatom content.

Why use it: Linkers strongly affect geometry, entropy, polarity, and binding presentation.

How it would be done with Python:
- Define the two fixed endpoint motifs.
- Create a library of linker fragments such as CH2, O, NH, CH2CH2, piperazine.
- Attach each linker between the same endpoints.
- Store linker identity as a separate annotation field.

Minimal pseudocode:

```
    linkers = ['CH2', 'O', 'NH', 'CH2CH2', 'piperazine']
    for linker in linkers:
        analog = connect(left_motif, linker, right_motif)
        save(analog, linker=linker)
```

7. Ring-Size And Ring-System Scans

What it is: Replace one ring with a related ring system such as pyrrolidine to piperidine or phenyl to pyridyl.

Why use it: Ring changes often shift potency, selectivity, and developability in a controlled way.

How it would be done with Python:
- Define a ring replacement table for each chemotype.
- Match the ring position in the parent scaffold.
- Swap in alternate ring systems while preserving vectors.
- Check resulting valence and geometry sanity before saving.

Minimal pseudocode:

```
    ring_swaps = ['pyrrolidine', 'piperidine', 'morpholine']
    for ring in ring_swaps:
        analog = replace_ring(parent, target_site='amine_ring', new_ring=ring)
        save(analog)
```

8. Heteroatom Walks

What it is: Move or swap heteroatoms within a scaffold to tune electronics, polarity, hydrogen bonding, and binding interactions.

Why use it: A heteroatom walk is a compact way to probe both binding and property shifts.

How it would be done with Python:
- Identify atom positions where carbon, nitrogen, oxygen, or sulfur swaps are reasonable.
- Generate all allowed single-site heteroatom variants.
- Discard unstable or chemically nonsensical products.
- Track which atom and position changed.

Minimal pseudocode:

```
    for position in editable_positions(parent):
        for atom in ['C', 'N', 'O', 'S']:
            analog = mutate_atom(parent, position, atom)
            if is_reasonable(analog):
                save(analog)
```

9. Scaffold Hopping

What it is: Keep the key pharmacophore pattern but replace the central core with a different scaffold.

Why use it: This is how a project escapes from one chemotype while preserving the core hypothesis.

How it would be done with Python:
- Define the required pharmacophore points or anchor vectors.
- Create a set of alternative cores that present those vectors similarly.
- Attach the same side chains to each alternate core.
- Cluster and compare the resulting chemotypes by novelty and properties.

Minimal pseudocode:

```
    for new_core in alternate_cores:
        hopped = transplant_sidechains(new_core, sidechains=parent_sidechains)
        save(hopped, strategy='scaffold_hop')
```

10. Fragment Growing

What it is: Start from a smaller active motif and extend it into nearby space with additional fragments.

Why use it: Fragment growing is useful when you know a minimal motif but need more interactions and potency.

How it would be done with Python:
- Choose a minimal active fragment or anchor motif.
- Define growth vectors and allowed fragment additions.
- Add one fragment at a time, then optionally iterate to a second growth round.
- Score each product for size, polarity, and tractability as the fragment grows.

Minimal pseudocode:

```
    seed = 'minimal_active_fragment'
    for frag in fragment_pool:
        analog = grow(seed, frag, vector='exit_1')
        save(analog)
```